PROSPEREUM (PSRE)

Developer Specification v2.1

0. Design Decisions Locked for v2.1

- **Epoch-based emission** (weekly), not per-claim.
- **Behavioral mining primitive**: *vault-executed PSRE buys only*.
- **PartnerVault SELL disabled** (no vault-based sells).
- **No price oracle, no USD normalization**.
- **Scarcity depends only on** $x = T/S_{emission}$.
- **70/30 split**: partners/stakers (LP shares staker pool).
- **Staker rewards are time-weighted** (anti flash-stake).
- **Partner status**: rolling EMA with tier multipliers.
- **No presale**; treasury seeds LP.

---

1. Constants & Global Parameters

1.1 Token supply

- S_TOTAL = 21_000_000e18
- S_EMISSION = 12_600_000e18

1.2 Epoch

- EPOCH_DURATION = 7 days
- genesisTimestamp set at deployment.

Epoch index:

- epochId = (block.timestamp - genesisTimestamp) / EPOCH_DURATION

1.3 Economic parameters (v1 defaults + bounds)

All parameters stored in RewardEngine contract.

- alphaBase (scaled 1e18):
  - default: 0.10e18
  - bounds: [0.05e18, 0.15e18]
- E0 (weekly scarcity ceiling, in PSRE wei):
  - default: 0.001 * S_EMISSION (i.e., 0.1% of emission reserve per week)
  - bounds: [0.0005*S_EMISSION, 0.002*S_EMISSION]
- k (scarcity exponent):

    - default: 2
    - **immutable** in v1
- clamp:
    - if `x >= 1` → `E_scarcity = 0`
    - use fixed-point to avoid rounding errors making it negative
- theta (EMA factor, scaled 1e18):
    - default: 1/13 ≈ 0.0769230769e18
    - **immutable** in v1 (or bounded but recommend immutable)

1.4 Tier parameters (share thresholds and multipliers)

Define rolling share s_p in 1e18.

Thresholds (defaults):

- BRONZE_TH = 0 (everyone at least bronze)
- SILVER_TH = 0.005e18 (0.5%)
- GOLD_TH = 0.02e18 (2.0%)

Multipliers (scaled 1e18):

- M_BRONZE = 1.0e18
- M_SILVER = 1.25e18
- M_GOLD = 1.6e18

These can be DAO-adjustable (bounded) or immutable. Recommend **bounded**.

1.5 Splits

- PARTNER_SPLIT = 0.70e18
- STAKER_SPLIT = 0.30e18
  DAO-bounded optional range:
- partner split in [0.60, 0.80]

1.6 rounding & dust rules

In partner rewards:

- compute reward_p with integer division
- total partner payouts may be slightly less than B_partners due to rounding
- same for stakers
- Rule: **do not mint dust.**
  Dust stays unminted (scarcity-positive), or you can carry it forward to next epoch budget (more complex).

1.7 **Keep "no vault sell" and "creditedNB" monotonic**

Coders need it explicit.

1.8 Developer-grade pseudo for minting

At finalizeEpoch(t):

1. compute B, split into B_partners, B_stakers
2. compute P_partners and P_stakers based on whether pools have eligible participants
3. P = P_partners + P_stakers
4. mint:

```
mintAmount = min(P, S_emission - T)

require(T + mintAmount <= S_emission)
mint(RewardEngine, mintAmount)
T += mintAmount
```

5. record epoch pools for claims

1.9 When sumR == 0

In early epochs there may be no partners.

In that case:

s_p = R_p / sumR

would divide by zero.

Add rule:

```
if sumR == 0:
  alpha_p = alpha_base
```

or skip status calculation entirely.

1.10 When computing partner rewards:

reward_p = B_partners * w_p / W

If

W == 0

then:

partner pool = 0

You partially stated this in the white paper but it should be explicit in the developer spec.

2. Contracts & Responsibilities

2.1 PSRE (ERC-20)

- Standard ERC-20 with decimals=18.
- mint(to, amount) callable only by RewardEngine.
- No other mint authority.

2.2 PartnerNFT (identity)

- Mintable by anyone (permissionless).
- 1 per partner entity (enforced by protocol rules if desired).
- **Non-transferable** or transfer-restricted recommended.
- Maps partnerId -> owner.

2.3 PartnerVaultFactory

- Creates a PartnerVault for a PartnerNFT owner.
- partnerId -> vaultAddress mapping.
- Ensures one vault per partnerId.

2.4 PartnerVault

Purpose: enforce accounting boundary for "provable buys".

Key rule: only buy() updates cumBuy.

State:

- uint256 cumBuy (PSRE received via vault swaps; cumulative since genesis)
- address partnerOwner (or partnerId)
- address rewardEngine (only engine can read via interface, or public view)

Functions:

- buy(...): swap (e.g., USDC→PSRE) and update cumBuy += psreOut
- distribute(to, amount): transfer PSRE out (no accounting changes)
- **NO sell()** in v1

Security:

- buy() must validate psreOut > 0
- Use ReentrancyGuard
- Restrict buy router calls to chosen router address set at vault deploy (partner-chosen) OR hardcode a router in vault template (safer).
  (This is not "protocol approval"; it's vault configuration.)

2.5 StakingVault (includes LP staking)

PSRE staking and LP staking are treated equivalently in the staking reward pool. No weighting multiplier is applied.

- Tracks time-weighted stake per epoch for each user.
- Supports staking PSRE and staking an LP token (e.g., PSRE/USDC LP).
- **Single StakingVault** with two staking assets:
    - stakePSRE(amount)
    - stakeLP(amount)
- Convert LP amount into **PSRE-equivalent**:

  stakeTime = stakeAmount × stakingDuration

2.6 RewardEngine (combined emission + reward vault)

The core monetary policy contract.

Responsibilities:

- Track total emitted T.
- Maintain epoch state.
- Compute ΔNB, EMA status, demand cap, scarcity cap, final budget B.
- Compute partner rewards and staker rewards.
- P-based minting rule (mint up to owed payouts subject to budget and reserve).
- Pay rewards.

---

3. Storage Layout (RewardEngine)

Global:

- uint256 public T;
- uint256 public genesisTimestamp;
- uint256 public lastFinalizedEpoch; (epochId)
- uint256 public alphaBase; (1e18)
- uint256 public E0; (wei)
- uint256 public k; (uint)
- uint256 public theta; (1e18)
- tier thresholds + multipliers
- split params

Partner accounting (by partnerId):

- uint256 cumBuySnapshot[partnerId] optional cache

- uint256 creditedNB[partnerId]
- uint256 R[partnerId] (rolling score, 1e18-scaled PSRE units)

Global rolling sum:

- uint256 sumR; (sum of all R values; updated incrementally)

Epoch reward records (optional for claim-based payout):

- mapping(epochId => bool finalized)
- mapping(epochId => uint256 B)
- mapping(epochId => uint256 partnersPool)
- mapping(epochId => uint256 stakersPool)
- partner reward owed: owedPartner[partnerId]
- staker reward owed: owedStaker[user]

Two payout designs:

- **Push**: engine sends rewards at finalize (not scalable if many users)
- **Pull** (recommended): finalize epoch computes pools; each participant claims later.

Given many stakers/partners, use **pull**.

So:

- finalizeEpoch() computes epoch pools and per-partner owed (partners count is limited).
- Stakers claim via claimStake(epochId) using snapshot of stakeTime.

---

4. Epoch Lifecycle

4.1 Functions

finalizeEpoch(uint256 epochId)

- Callable by anyone after epoch ends.
- Finalizes exactly one epoch at a time: epochId == lastFinalizedEpoch + 1.
- Computes budgets and records pools.

claimPartner(uint256 epochId, uint256 partnerId)

- Transfers owed partner reward for epochId.

claimStake(uint256 epochId)

- Transfers owed staker reward for epochId.

(If you want to simplify: claimAll() loops across epochs but careful with gas.)

---

5. Detailed Algorithms

5.1 Partner ΔNB computation

Inputs:

- partnerVault = vaultOf(partnerId)
- cumBuy = PartnerVault.cumBuy()

Definitions:

- NB = cumBuy (since sell disabled)
- deltaNB = max(0, NB - creditedNB[partnerId])
- update: creditedNB[partnerId] += deltaNB

Rounding: integers in wei.

5.2 Rolling EMA update (status)

We compute:

- R_new = (1-θ)*R_old + θ*deltaNB

All in integer fixed-point.

Implementation:

- store R in wei units (same unit as PSRE) OR 1e18-scaled.
  Simplest: keep in wei and apply θ as 1e18 fixed-point:

R_new = (R_old * (1e18 - theta) + deltaNB * theta) / 1e18

Update sumR:

- sumR = sumR - R_old + R_new

Then compute share:

- s = (R_new * 1e18) / sumR (if sumR > 0)

Tier multiplier:

if s >= GOLD_TH: m = M_GOLD
else if s >= SILVER_TH: m = M_SILVER
else m = M_BRONZE

Alpha:

- alpha_p = (alphaBase * m) / 1e18

5.3 Demand cap

For the epoch being finalized:

$$E_{demand} = \sum_{p} \alpha_p \cdot \Delta NB_p$$

Implementation (fixed-point):

- alpha_p is 1e18 scaled.
- deltaNB_p is wei.
- alpha_p * deltaNB_p / 1e18.

Sum over partners who have deltaNB_p > 0 (only those matter).

5.4 Scarcity cap f(x)

Definitions:

- x = T / S_EMISSION (both wei) in 1e18 fixed-point:
    - x = (T * 1e18) / S_EMISSION

Use:

$$E_{scarcity} = E0 \cdot (1 - x)^k$$

Implementation (k=2 in v1):

- oneMinusX = 1e18 - x (clamp at 0)
- for k=2:
    - (1-x)^2 = oneMinusX * oneMinusX / 1e18
- E_scarcity = E0 * (1-x)^k / 1e18

Clamp: if x >= 1e18, E_scarcity = 0.

5.5 Final budget B

- B = min(E_demand, E_scarcity)

5.6 Split

- B_partners = B * PARTNER_SPLIT / 1e18
- B_stakers = B - B_partners (avoid rounding drift)

5.7 Partner reward distribution

Weight per partner:

- w_p = alpha_p * deltaNB_p / 1e18 (wei)

Total weight:

- W = Σ w_p

Partner epoch reward:

- reward_p = B_partners * w_p / W

If W==0, partner pool becomes 0 and can be carried forward or left unminted. (Recommend: set partner pool=0.)

5.8 Staker reward distribution (pull-based)

StakingVault must expose:

- totalStakeTime(epochId)
- stakeTimeOf(user, epochId)

Then:

- reward_i = B_stakers * stakeTime_i / totalStakeTime

If totalStakeTime==0, staker pool becomes 0.

---

6. Minting

- Compute P_partners and P_stakers (0 if pool empty)
- P = P_partners + P_stakers
- Mint:
  - mintAmount = min(P, S_emission - T)
  - Then payouts come from the minted amount (and any existing leftover balance)

---

7. StakingVault: time-weight accounting

Each deposit/withdraw updates user accumulator.

Maintain for each user:

- balance
- lastUpdateTimestamp
- accStakeTime for current epoch

On any action:

accStakeTime += balance * (now - lastUpdateTimestamp)
lastUpdateTimestamp = now

At epoch boundary:

- snapshot stakeTime for that epoch and reset accumulator (or maintain per-epoch mapping).

Implement similarly for LP stake with weight factor.

---

8. Anti-Exploitation Constraints

- PartnerVault cannot sell PSRE in v1.
- creditedNB monotonic, cannot decrease.
- Only vault-executed buys count.
- EMA update uses deltaNB only (not cumulative).

- One epoch finalized at a time, strictly sequential.
- Time-weighted staking prevents flash stake.
- No price oracle avoids oracle manipulation.

---

9. Governance (DAO / multisig) Controls

DAO/multisig can adjust (bounded):

- alphaBase within [0.05,0.15]
- E0 within a bounded range
- tier thresholds and multipliers (bounded)
- split ratio within [0.60,0.80]

mint outside finalizeEpoch P-based minting rule

Add a timelock for any parameter updates.

---

10. Events (required)

Emit events for indexers and audits:

- EpochFinalized(epochId, B, E_demand, E_scarcity, B_partners, B_stakers, minted)
- PartnerDeltaComputed(epochId, partnerId, deltaNB, alpha_p, weight, reward)
- StakeClaimed(epochId, user, stakeTime, reward)
- PartnerClaimed(epochId, partnerId, reward)
- PartnerBought(partnerId, amountIn, psreOut, cumBuy)
- Distributed(partnerId, to, amount)

---

11. Required Invariants (assertions)

Always enforce:

- T <= S_EMISSION
- creditedNB[p] <= cumBuy[p]
- deltaNB >= 0
- B <= E_demand and B <= E_scarcity
- mintAmount <= P
  mintAmount <= (S_emission - T)
- sumPartnerRewards <= B_partners (allow dust remainder)
- sumStakeRewards <= B_stakers (allow dust remainder)

---

12. Gas/Scalability Notes

- Partners are likely limited in count → partner reward computation can be done in finalize.
- Stakers can be large → staking rewards must be **pull-based** using stakingVault snapshots.
- Avoid looping over all stakers in RewardEngine.

---

13. v1 Implementation Checklist (what coder should build)

1. ERC-20 PSRE with mint restricted to RewardEngine
2. PartnerNFT + PartnerVaultFactory + PartnerVault buy/distribute
3. StakingVault with time-weighted accounting (PSRE + optional LP)
4. RewardEngine:
    - finalizeEpoch
    - compute partner ΔNB and EMA
    - compute budgets, compute P, mint up to P (bounded by scarcity and remaining reserve).
    - record epoch pools
    - claimPartner / claimStake
    - bounded parameter governance
5. Unit tests:
    - creditedNB monotonicity
    - epoch sequencing
    - scarcity function correctness near cap
    - time-weight staking anti-flash
    - rounding/dust behavior